Module 11 - File Handling

Table of Contents

- [ ] 1. Why File Handling Matters in Cybersecurity
- [ ] 2. Opening and Closing Files — `open()` and `close()`
- [ ] 3. The `with` Statement — The Right Way to Open Files
- [ ] 4. Reading Files — `read()`, `readline()`, `readlines()`
- [ ] 5. Writing and Appending — `'w'` and `'a'` Modes
- [ ] 6. File Modes Reference
- [ ] 7. Path Handling with `os.path`
- [ ] 8. Listing and Walking Directories
- [ ] 9. Summary and Next Steps

---

1. Why File Handling Matters in Cybersecurity

Almost everything a security professional does involves files:

Task	Files involved
SOC analyst investigating an incident	Web server logs, auth logs, firewall logs
Pentester documenting findings	Scan output files, vulnerability reports
Security engineer deploying a tool	Config files (JSON/YAML), allow/blocklists
Threat intelligence	IOC feeds (CSV/TXT), STIX bundles
Compliance officer	Evidence files, audit logs

In short: if you cannot read, parse, and write files in Python, you cannot automate any real security workflow.

The `data/` folder for these exercises

All exercises in this notebook use files from the `data/` directory next to this notebook:

```
data/
  access.log          Web server access log
  alerts.txt          Pipe-delimited SIEM alert export
  blocked_ips.txt     Threat intelligence IP blocklist
```

```
vulnerabilities.csv Vulnerability scan results
config.json         Security tool configuration
```

---

2. Opening and Closing Files — `open()` and `close()`

Opening a file in Python gives you a **file object** — a handle that lets you read from or write to the file. Think of it like opening a safe: once you're done, you must close it properly, or data can be lost and resources leak.

```
file = open('path/to/file', 'mode')
# ... do things with file ...
file.close()    # always required — but easy to forget
```

The **first argument** is the file path (relative or absolute). The **second argument** is the mode — what you intend to do with the file.

Relative vs. Absolute paths

```
Relative path:   data/access.log        (relative to where the
script runs)
Absolute path:   /home/analyst/data/access.log  (full path from
filesystem root)
```

Best practice: use relative paths in notebooks so they work on any machine. Use `os.path` or `pathlib` to build paths safely — never hardcode `/home/yourname/...`.

In [ ]:
```
# Topic: Basic open/close — reading a blocklist
# This is the manual approach — section 3 shows the better way

blocklist_path = "data/blocked_ips.txt"

# Open the file in read mode
blocklist_file = open(blocklist_path, "r")

# Read the entire content as one big string
content = blocklist_file.read()

# CRITICAL: always close — if you forget, the OS may keep the file locked
# and changes may not be flushed to disk
blocklist_file.close()

print(content)
```

⚠️ Warning — Never Forget to Close

If your program crashes between `open()` and `close()`, the file never gets closed. This means:

- On Windows: the file may be locked, preventing other processes from reading it
- Data written in `'w'` or `'a'` mode may not be flushed to disk
- In long-running security tools (daemons, scanners): resource leaks accumulate

The solution is the `with` statement — section 3.

---

3. The `with` Statement — The Right Way to Open Files

The `with` statement is a **context manager**. It guarantees the file is closed automatically when the indented block exits — even if an exception occurs.

```
with open('file.txt', 'r') as f:
    content = f.read()
# file is automatically closed here — no .close() needed
```

You should use `with` for **every single file operation**. The manual `open()` / `close()` pattern exists but should be avoided.

The `as f` part gives the file object a name (`f` is a common short name, but descriptive names like `log_file` or `blocklist_file` are better in real tools).

In [ ]:
```
# Topic: with statement — the correct way to read a security log

log_path = "data/access.log"

with open(log_path, "r") as log_file:
    content = log_file.read()
# file is already closed here

print(f"Log file contains {len(content)} characters")
print(f"First 200 characters:")
print(content[:200])
```

🎯 Practice — `with` Statement

Exercise 1 — Write

Open `data/alerts.txt` using a `with` statement and read its content. Count how many lines the file contains (hint: split on newlines or use `readlines()`). Print the total line count and the first line.

In [ ]:
```
# Your code here
```

▶ 💡 **SOLUTION**

---

4. Reading Files — `read()`, `readline()`, `readlines()`

Python gives you three ways to read a file, each suited to different situations:

Method	Returns	Best for
`f.read()`	One big string (entire file)	Small files, when you need the full content
`f.readline()`	One line as a string	Processing line by line manually
`f.readlines()`	List of strings, one per line	When you need all lines in a list
`for line in f:`	One line per iteration	**Best for large files** — reads lazily

Security note on large log files

Production log files can be gigabytes in size. Using `f.read()` on a 2 GB log file loads 2 GB into RAM at once — that will crash a tool or slow a scan to a halt.

For large files, always iterate line by line with `for line in f:`.

In [ ]:
```
# Topic: read() — entire file as one string
# Fine for small config files or blocklists

with open("data/blocked_ips.txt", "r") as blocklist_file:
    full_content = blocklist_file.read()

print("--- Full content (read()) ---")
print(full_content)
print(f"Type: {type(full_content)}")
```

In [ ]:
```
# Topic: readlines() — all lines as a list
# Each element is one line, including the trailing newline character \n

with open("data/blocked_ips.txt", "r") as blocklist_file:
    all_lines = blocklist_file.readlines()

print("--- All lines (readlines()) ---")
for i, line in enumerate(all_lines):
    print(f"  Line {i}: {repr(line)}")  # repr() shows the \n explicitly

print(f"\nType: {type(all_lines)}")
print(f"Count: {len(all_lines)} lines")
```

In [ ]:
```
# Topic: for line in f — best practice for large files
# Reads one line at a time — low memory usage
# .strip() removes the trailing \n from each line

print("--- Iterating line by line ---")

blocked_ips = []

with open("data/blocked_ips.txt", "r") as blocklist_file:
    for line in blocklist_file:
        line = line.strip()              # remove leading/trailing whitespace and \

        if not line or line.startswith("#"):  # skip empty lines and comments
            continue

        ip = line.split(" | ")[0]        # extract just the IP address
        blocked_ips.append(ip)

print(f"Blocked IPs loaded: {blocked_ips}")
```

🎯 Practice — Reading Files

Exercise 2 — Write

Read `data/alerts.txt` line by line. Each line has the format: `ALT-001|2024-03-15 08:13:19|185.220.101.47|SSH_BRUTE_FORCE|HIGH`

Build a list called `critical_alerts` containing only the alert IDs (first field) where the severity (last field) is `'CRITICAL'`. Print the count and the list.

In [ ]:
```
# Your code here
```

▶ 💡 **SOLUTION**

Exercise 3 — Predict

Before running the cell, predict:

1. What type does `lines` have?
2. What does `lines[0]` end with — a newline `\n` or not?
3. What does `lines[0].strip()` return vs `lines[0]`?

In [ ]:
```
# Predict first, then run!

with open("data/blocked_ips.txt", "r") as f:
    lines = f.readlines()

print(type(lines))           # 1. what type?
print(repr(lines[1]))        # 2. does it end with \n? (line 1 = first non-comment
print(repr(lines[1].strip())) # 3. after strip?
```

---

5. Writing and Appending — `'w'` and `'a'` Modes

Mode	Behaviour	File exists?	File missing?
`'w'`	Write (overwrite)	**Wipes the file completely**	Creates it
`'a'`	Append (add to end)	Adds after existing content	Creates it
`'x'`	Exclusive create	Raises `FileExistsError`	Creates it

⚠️ Warning — `'w'` destroys existing data

As soon as you `open(path, 'w')`, the file is truncated to zero bytes **immediately**, even before you write anything. If you open the wrong file with `'w'`, your data is gone.

For audit logs and security event files: always use `'a'` (append). You should never overwrite a security log — that would destroy evidence.

In [ ]:
```
# Topic: 'w' mode — writing a new findings report
# Use 'w' when you want to generate a fresh output file from scratch

import os

report_path = "data/scan_report.txt"

# Write the report header
with open(report_path, "w") as report_file:
    report_file.write("=== Vulnerability Scan Report ===\n")
    report_file.write("Date: 2024-03-15\n")
    report_file.write("Target: 10.0.0.0/24\n")
    report_file.write("\n")
    report_file.write("FINDINGS:\n")

print(f"Report created: {report_path}")

# Verify it was written
with open(report_path, "r") as f:
    print(f.read())
```

In [ ]:
```
# Topic: 'a' mode — appending findings to the report
# 'a' is safe: it never destroys existing content

findings = [
    "10.0.0.5  | PORT 3389 OPEN  | RDP exposed to internet — HIGH risk",
    "10.0.0.12 | PORT 23 OPEN    | Telnet service running — CRITICAL risk",
    "10.0.0.18 | CVE-2021-44228  | Log4Shell — unpatched — CRITICAL",
]

with open(report_path, "a") as report_file:
```

```
    for finding in findings:
        report_file.write(finding + "\n")  # \n = newline between each finding

# Read the complete report to verify
with open(report_path, "r") as f:
    print(f.read())
```

🎯 Practice — Writing Files

Exercise 4 — Write

Read `data/access.log` and find all lines where the HTTP status code is `401` (failed authentication attempt).

Write those lines to a new file `data/failed_logins.txt`. Include a header line: `# Failed login attempts extracted from access.log`.

Then read it back and print the content to confirm.

In [ ]:
```
# Your code here
```

▶ 💡 **SOLUTION**

Exercise 5 — Write

Simulate an audit log. Create a function `log_event(filepath, event_type, source_ip, details)` that:

- Appends a single line to the file in the format: `[TIMESTAMP] | EVENT_TYPE | SOURCE_IP | DETAILS`
- Uses `'a'` mode so existing entries are never overwritten
- The timestamp should use `import time; time.strftime('%Y-%m-%d %H:%M:%S')`

Call it 3 times with different events and then read the file back.

In [ ]:
```
# Your code here
import time
```

▶ 💡 **SOLUTION**

---

6. File Modes Reference

Quick reference for all modes:

Mode	Read	Write	Position	File exists	File missing
`'r'`	Yes	No	Start	OK	`FileNotFoundError`
`'w'`	No	Yes	Start	**Truncates to zero**	Creates
`'a'`	No	Yes	End	Appends	Creates
`'x'`	No	Yes	Start	`FileExistsError`	Creates
`'r+'`	Yes	Yes	Start	OK	`FileNotFoundError`
`'rb'`	Yes	No	Start	OK (binary)	`FileNotFoundError`
`'wb'`	No	Yes	Start	**Truncates** (binary)	Creates

When to use which mode in security work

Scenario	Mode
Reading a log file	`'r'`
Creating a fresh scan report	`'w'`
Adding entries to an audit log	`'a'`
Creating a new file, fail if it exists	`'x'` (safe — prevents accidental overwrite)
Reading a binary (executable, PCAP)	`'rb'`

---

7. Path Handling with `os.path`

The `os` module provides tools for working with file system paths in a way that works on Windows, Linux, and macOS — critical when your tool runs in multiple environments.

The most important functions:

Function	What it does	Example
`os.path.join(a, b)`	Build a path safely	`os.path.join('data', 'logs', 'access.log')`
`os.path.exists(path)`	Check if path exists	`True` or `False`
`os.path.isfile(path)`	Check if it's a file	`True` or `False`
`os.path.isdir(path)`	Check if it's a directory	`True` or `False`
`os.path.basename(path)`	Get filename only	`'access.log'`

Function	What it does	Example
`os.path.dirname(path)`	Get directory only	`'/var/log/nginx'`
`os.path.abspath(path)`	Get full absolute path	`'/home/analyst/data/access.log'`
`os.path.splitext(path)`	Split name and extension	`('access', '.log')`

Why `os.path.join()` instead of string concatenation

```
# BAD: breaks on Windows (which uses \ not /)
path = "data/" + "logs/" + "access.log"

# GOOD: works on all operating systems
path = os.path.join("data", "logs", "access.log")
```

In [ ]:
```
# Topic: os.path — building and inspecting paths safely

import os

# Build paths portably
data_dir = os.path.join("data")
log_path = os.path.join(data_dir, "access.log")
alert_path = os.path.join(data_dir, "alerts.txt")
missing_path = os.path.join(data_dir, "doesnt_exist.log")

print("=== Path inspection ===")
print(f"Log path:         {log_path}")
print(f"Absolute path:    {os.path.abspath(log_path)}")
print(f"Directory:        {os.path.dirname(os.path.abspath(log_path))}")
print(f"Filename:         {os.path.basename(log_path)}")
print(f"Name, extension:  {os.path.splitext(os.path.basename(log_path))}")

print("\n=== Existence checks ===")
print(f"{log_path} exists:   {os.path.exists(log_path)}")
print(f"{missing_path} exists: {os.path.exists(missing_path)}")
print(f"{log_path} is file:  {os.path.isfile(log_path)}")
print(f"{data_dir} is dir:   {os.path.isdir(data_dir)}")
```

In [ ]:
```
# Topic: safe file opening — always check before opening in write mode
# Prevents accidentally overwriting a file you didn't mean to touch

import os

def safe_open_for_writing(filepath):
    """Open a file for writing, but warn if it already exists."""
    if os.path.exists(filepath):
        # File already exists — make a decision before overwriting
        size_kb = os.path.getsize(filepath) / 1024
        print(f"[WARN] {filepath} already exists ({size_kb:.1f} KB) — overwriting")
```

```
    return open(filepath, "w")


# Test: the report file we created in section 5 already exists
with safe_open_for_writing("data/scan_report.txt") as f:
    f.write("Refreshed report content\n")

print("Done.")
```

🎯 Practice — `os.path`

Exercise 6 — Write

Write a function `file_summary(filepath)` that:

1. Checks if the file exists — if not, prints an error and returns `None`
2. Uses `os.path` to print: the filename, the directory, the file extension, the size in bytes (`os.path.getsize()`)
3. Opens the file and counts the number of lines
4. Returns a dict with keys `filename`, `extension`, `size_bytes`, `line_count`

Test with `file_summary('data/access.log')` and `file_summary('data/missing.txt')`.

In [ ]:
```
# Your code here
import os
```

▶ 💡 **SOLUTION**

---

8. Listing and Walking Directories

Two useful `os` functions for working with directories:

Function	What it does
`os.listdir(path)`	Returns list of filenames in a directory (one level only)
`os.walk(path)`	Recursively walks all subdirectories, yielding `(dirpath, dirs, files)`

In security work, `os.walk()` is used to scan entire directory trees for log files, suspicious executables, or to aggregate reports.

In [ ]:
```
# Topic: os.listdir() — what's in our data directory?

import os

data_dir = "data"
```

```
all_items = os.listdir(data_dir)
print(f"All items in '{data_dir}':")
for item in all_items:
    full_path = os.path.join(data_dir, item)
    item_type = "[DIR] " if os.path.isdir(full_path) else "[FILE]"
    print(f"  {item_type} {item}")
```

In [ ]:
```
# Topic: filtering by extension — find all .log files
# Useful for log aggregation tools

import os

data_dir = "data"
log_files = []

for filename in os.listdir(data_dir):
    _, ext = os.path.splitext(filename)
    if ext == ".log":
        full_path = os.path.join(data_dir, filename)
        log_files.append(full_path)

print(f"Log files found: {len(log_files)}")
for f in log_files:
    print(f"  {f}  ({os.path.getsize(f)} bytes)")
```

In [ ]:
```
# Topic: os.walk() — recursively walk a directory tree
# Useful when logs are spread across nested subdirectories

import os

print("All files under 'data/' (recursive):")

for dirpath, subdirs, files in os.walk("data"):
    # dirpath: current directory being examined
    # subdirs: list of subdirectory names inside dirpath
    # files:   list of filenames inside dirpath
    for filename in files:
        full_path = os.path.join(dirpath, filename)
        size = os.path.getsize(full_path)
        print(f"  {full_path}  ({size} bytes)")
```

🎯 Practice — Directory Operations

Exercise 7 — Write

Write a function `find_files_by_extension(directory, extension)` that:

- Uses `os.listdir()` to find all files with the given extension in the directory
- Returns a list of full paths (use `os.path.join`)

Then write `count_total_lines(file_list)` that:

- Takes the list of paths from above
- Opens each file and counts total lines across all files
- Returns the total count

Test: find all `.txt` files in `data/` and count their total lines.

In [ ]:
```
# Your code here
import os
```

▶ 💡 **SOLUTION**

---

9. Summary and Next Steps

Key Rules

Rule	Why it matters
Always use `with open(...)`	Guarantees the file closes even on errors
Use `'a'` for audit/security logs	`'w'` destroys evidence
Check `os.path.exists()` before writing	Prevents accidental overwrites
Iterate with `for line in f:` for large files	Avoids loading GBs into RAM
Use `os.path.join()` for paths	Ensures cross-platform compatibility
Always `.strip()` lines after reading	Removes the trailing `\n`

What comes next

01_Advanced_File_Handling.ipynb — CSV module, JSON module, encoding, `pathlib`, creating/deleting directories